In [2]:
```
library(ggplot2)
library(mgcv)
```

```
Loading required package: nlme

This is mgcv 1.8-31. For overview type 'help("mgcv-package")'.
```

C3M4 Peer Reviewed Assignment

Outline:

The objectives for this assignment:

1. Observe the difference between GAMs and other regression models on simulated data.
2. Review how to plot and interpret the coefficient linearity for GAM models.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

In [3]:
```
# Load packages
library(ggplot2)
library(mgcv)
```

Problem 1: GAMs with Simulated Data

In this example, we show how to check the validity of a generalized additive model (GAM) (using the `gam()` function) using simulated data. This allows us to try and understand the intricacies of `gam()` without having to worry about the context of the data.

1. (a) Simulate the Data

Let $n = 200$. First, construct three predictor variables. The goal here is to construct a GAM with different types of predictor terms (e.g., factors, continuous variables, some that will enter linearly/parametrically, some that enter transformed).

1. x1: A continuous predictor that, we will suppose has a nonlinear relationship with the response.
2. x2: A categorical variable with three levels: `s`, `m`, and `t`.
3. x3: A categorical variable with two levels: `TRUE` and `FALSE`.

Then, make the response some nonlinear/nonparametric function of $\mathbf{x}$. For our case, use:

$$\log(\mu_i) = \beta_1 + \sin(0.5x_{i,1}^2) - x_{i,2} + x_{i,3}$$

This model is a Poisson GAM. In a realworld situation, we wouldn't know this functional relationship and would estimate it. Other terms are modeled parametrically. The response has normal noise.

Note that:

1. The construction of $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_n)^T$ has the linear predictor exponentiated, because of the nature of the link function.
2. We use $\boldsymbol{\mu}$ to construct $\mathbf{y} = (y_1, \ldots, y_n)^T$. The assumption for Poisson regression is that the random variable Y_i that generates y_i is Poisson with mean μ_i.
3. `as.integer(as.factor(VARIABLE))` converts the labels of VARIABLE to 1, 2, 3,.. so that we can construct the relationship for these factors.

Plot the relationship of $\mathbf{y}$ to each of the predictors. **Then, split the data into a training (`train_sim`) and test (`test_sim`) set**.

In [4]:
```
set.seed(0112358)
# n = number of data points
n = 200

df = data.frame(
    x1 = rnorm(n, mean = 25, sd = 5),
    x2 = as.factor(sample(c("s", "m", "t"), size = n, replace = TRUE)),
    x3 = sample(c(T, F), size = n, replace =TRUE)
)

head(df)

df$mean = exp(log(0.5 * df$x1 ^ 2) - as.integer(as.factor(df$x2)) + as.integer(as.factor(df
$x3)))

df$y = rpois(n, df$mean)

head(df)
summary(df)

ggplot(df, aes( x = x1, y = y)) +
    geom_point()

eighty = floor(0.8 * nrow(df))
index = sample(seq_len(nrow(df)), size = eighty)

train_sim = df[index,]
test_sim = df[-index,]

cat("There should be ", eighty," records in train set. There are ", dim(train_sim)[1], "recor
ds.")
```

A data.frame: 6 × 3

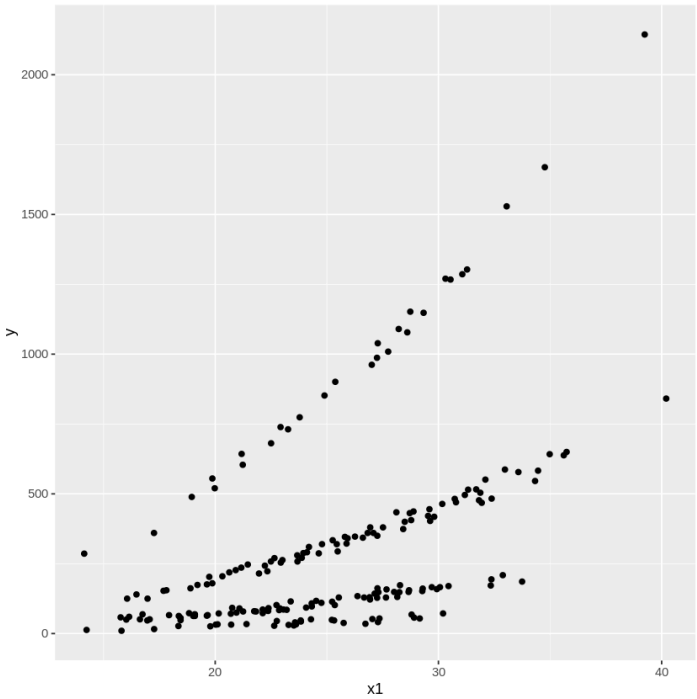
	x1 <dbl>	x2 <fct>	x3 <lgl>
1	22.64409	t	FALSE
2	30.73000	m	FALSE
3	27.65025	s	FALSE
4	23.05748	s	FALSE
5	31.28419	m	TRUE
6	15.80383	t	FALSE

A data.frame: 6 × 5

	x1 <dbl>	x2 <fct>	x3 <lgl>	mean <dbl>	y <int>
1	22.64409	t	FALSE	34.69690	28
2	30.73000	m	FALSE	472.16642	482
3	27.65025	s	FALSE	140.62855	129
4	23.05748	s	FALSE	97.79110	86
5	31.28419	m	TRUE	1330.19193	1303
6	15.80383	t	FALSE	16.90074	10

```
       x1         x2         x3               mean               y         
 Min.   :14.13   m:64   Mode :logical   Min.   :  13.72   Min.   :  10.0  
 1st Qu.:21.18   s:76   FALSE:105       1st Qu.:  80.39   1st Qu.:  74.5  
 Median :24.77   t:60   TRUE :95        Median : 168.44   Median : 168.0  
 Mean   :24.99                          Mean   : 309.11   Mean   : 308.5  
 3rd Qu.:28.67                          3rd Qu.: 414.88   3rd Qu.: 418.8  
 Max.   :40.20                          Max.   :2092.83   Max.   :2144.0  
```

```
There should be  160  records in train set. There are  160 records.
```



1. (b) Other Regression Models

Before jumping straight into GAMs, let's test if other regression models work. What about a regular linear regression model with ordinary least squares, and a generalized linear model for Poisson regression?

First fit a linear regression model to your `train_sim` data. We know that all of the predictors were used to make the response, but are they all significant in the linear regression model? Explain why this may be.

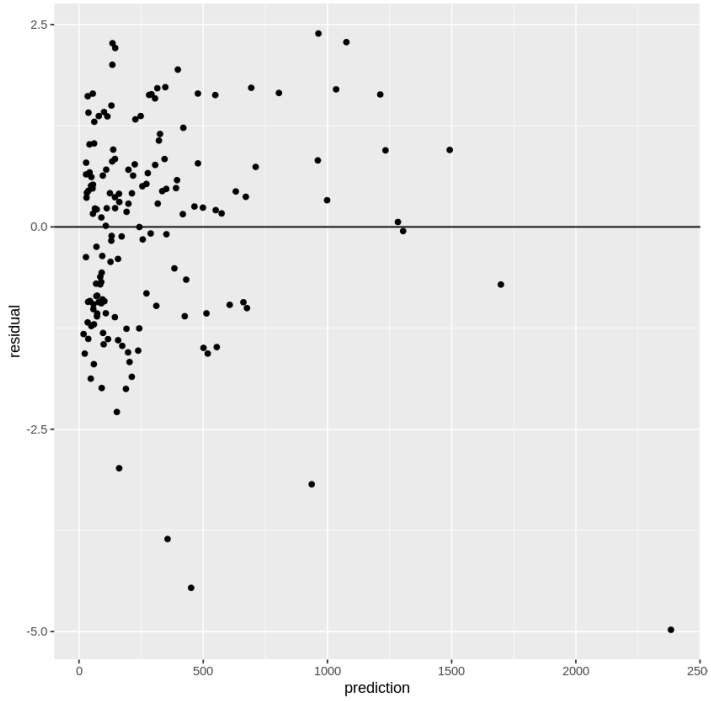
Then fit a Generalize Linear Model (GLM) to the `train_sim` data. Plot three diagnostic plots for your GLM:

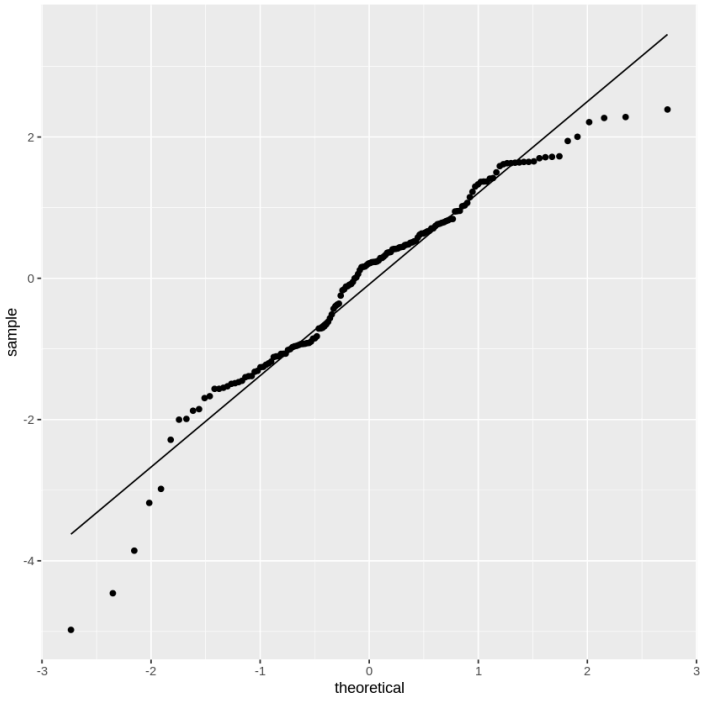
1. Residual vs. log(Fitted Values)
2. QQPlot of the Residuals
3. Actual Values vs. Fitted Values

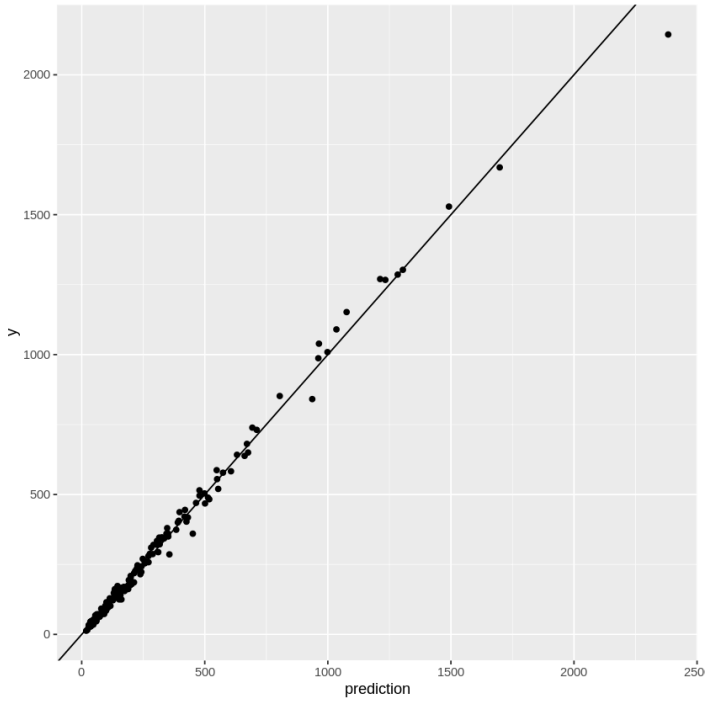
Using these plots, determine whether this model is a good fit for the data. Make sure to explain your conclusions and reasoning.

In [5]:
```
# Fit a LM model to the data

lm_train = lm(y ~ x1 + x2 + x3, data = train_sim)
summary(lm_train)

# Fit a GLM model to the data

glm_train = glm(y ~ x1 + x2 + x3, data = train_sim, family = "poisson")
summary(glm_train)

#residual plot

residual = residuals(glm_train, type = "deviance")
prediction = predict(glm_train, type = "response")

results = data.frame(y = train_sim$y, prediction, residual)

# Create the three specified plots

ggplot(results, aes(prediction, residual))+
    geom_point() +
    geom_hline(yintercept = 0)


## qqplot

ggplot(results, aes(sample = residual))+
    stat_qq() + stat_qq_line()

#fitted vs actual

ggplot(results, aes(prediction,y))+
    geom_point()+
    geom_abline(slope = 1)
```

```
Call:
lm(formula = y ~ x1 + x2 + x3, data = train_sim)

Residuals:
    Min      1Q  Median      3Q     Max
-222.84 -120.08  -43.77   76.42 1005.34

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -213.056     77.549  -2.747  0.00672 **
x1            27.064      2.758   9.814  < 2e-16 ***
x2s         -369.864     33.772 -10.952  < 2e-16 ***
x2t         -507.994     36.019 -14.104  < 2e-16 ***
x3TRUE       289.721     27.940  10.370  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 175 on 155 degrees of freedom
Multiple R-squared:  0.7598,    Adjusted R-squared:  0.7536
F-statistic: 122.6 on 4 and 155 DF,  p-value: < 2.2e-16

Call:
glm(formula = y ~ x1 + x2 + x3, family = "poisson", data = train_sim)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-4.9778  -0.9578   0.2117   0.7874   2.3898

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.7993224  0.0251780  150.90   <2e-16 ***
x1           0.0756880  0.0008212   92.17   <2e-16 ***
x2s         -0.9951120  0.0107563  -92.52   <2e-16 ***
x2t         -1.9660893  0.0177880 -110.53   <2e-16 ***
x3TRUE       1.0066777  0.0097026  103.75   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 48258.25  on 159  degrees of freedom
Residual deviance:   266.04  on 155  degrees of freedom
AIC: 1397.9

Number of Fisher Scoring iterations: 4
```







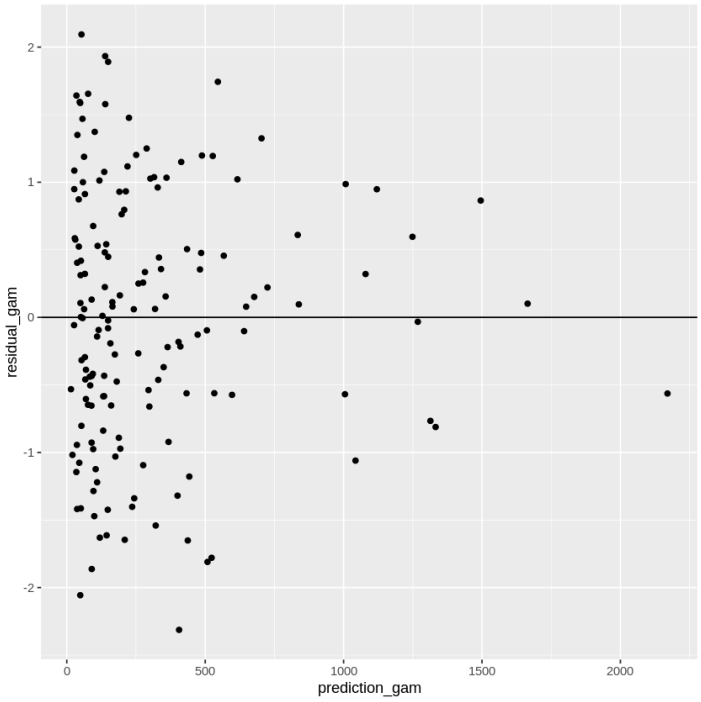
In the linear model, all of the xi are statisticall significant, even when we know it's not a linear relationship.

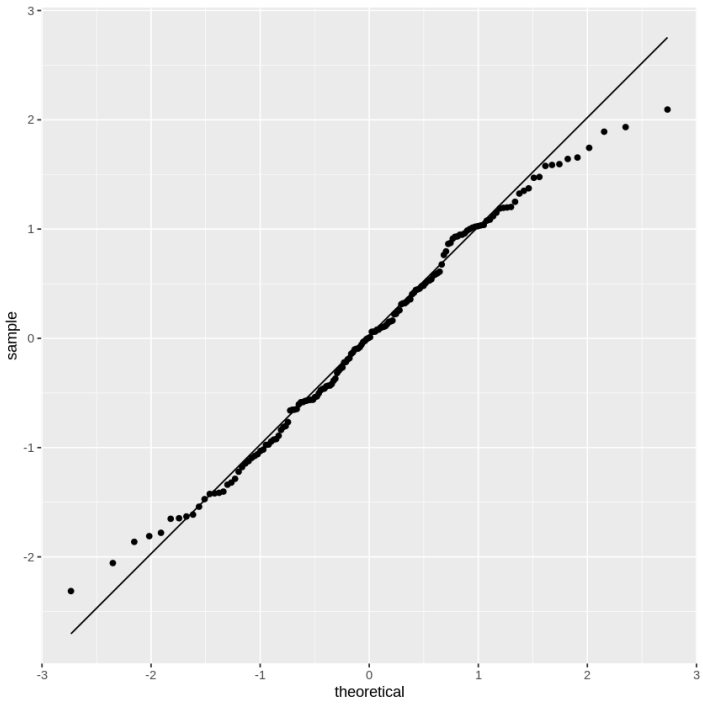
In the plots, they all could be better specially on the QQ and the observed vs predicted values.

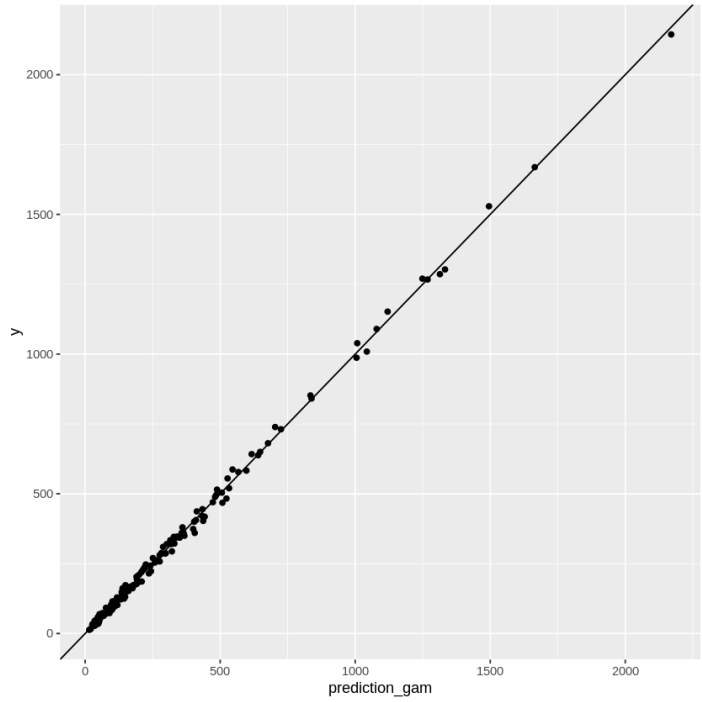
1. (c) Looking for those GAMs

Now, it's time to see how a generalized additive model (GAM) performs! Fit a GAM to the data. Construct the same three plots for your GAM model. Do these plots look better than those of the GLM?

In [6]:
```
# Fit a GAM model to the data

gam_train = gam(y ~ s(x1) + x2 + x3, data = train_sim, family = poisson)

# Construct the three specified plots


residual_gam = residuals(gam_train, type = "deviance")
prediction_gam = predict(gam_train, type = "response")

results_gam = data.frame(y = train_sim$y, prediction_gam, residual_gam)

#residual vs fitted

ggplot(results_gam, aes(prediction_gam, residual_gam))+
    geom_point() +
    geom_hline(yintercept = 0)


## qqplot

ggplot(results_gam, aes(sample = residual_gam))+
    stat_qq() + stat_qq_line()

#fitted vs actual

ggplot(results_gam, aes(prediction_gam,y))+
    geom_point()+
    geom_abline(slope = 1)
```


residual_gam
prediction_gam
2
1
0
-1
-2
0
500
1000
1500
2000


sample
theoretical
3
2
1
0
-1
-2
-3
-2
-1
0
1
2
3



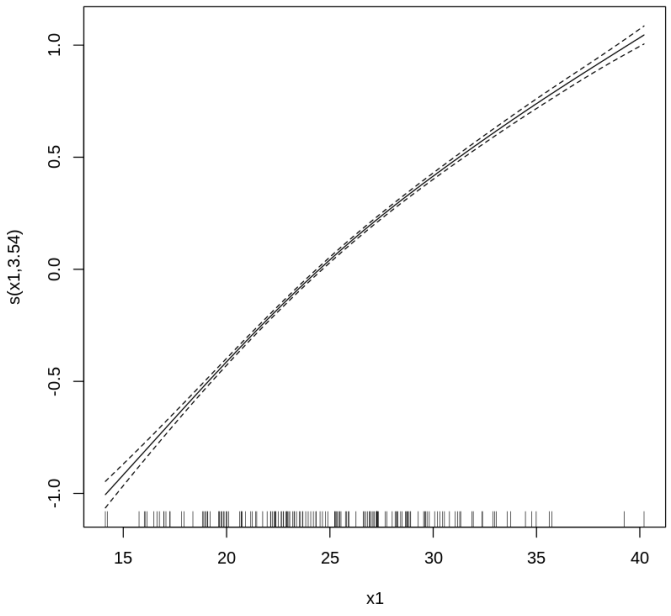
The plots look a lot better, specially the QQ Plot. The fit appears to be a lot better. On the predicted vs actual, even the largest values are closer to the x = y line.

1. (d) Interpreting GAMs

We made a GAM model! However GAMs are harder to interpret than regular linear regression models. How do we determine if a GAM model was necesary? Or, in other words, how do we determine if our predictors have a linear relationship with the response?

Use the `plot.gam()` function in the mgcv library to plot the relationship between `y` and `x1`. Recall that x_1 entered our model as $\sin(0.5x_{i,1}^2)$, and we plotted that relationship in **1.(a)**. Does your plot confirm this relationship?

In [7]:
```
plot.gam(gam_train)
```



Yes, the plot confirms the relationship between x1 and y.

1.(e) Model comparison

Compute the mean squared prediction error (MSPE) for each of the three models above (regression model, GLM, and GAM). State which model performs based according to this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k}\sum_{i=1}^{k}\left(y_i^\star - \hat{y}_i^\star\right)^2$$

where $y_i^\star$ are the observed response values in the test set and $\hat{y}_i^\star$ are the predicted values for the test set (using the model fit on the training set).

In [8]:
```
#mspe for lm

lm_pred = predict(lm_train, newdata = test_sim)
lm_mspe = mean((test_sim$y - lm_pred) ^ 2)

#mspe for glm

glm_pred = predict(glm_train, newdata = test_sim, type = "response")
glm_mspe = mean((test_sim$y - glm_pred) ^ 2)

# mspe for gam

gam_pred = predict(gam_train, newdata = test_sim, type = "response")
gam_mspe = mean((test_sim$y - gam_pred) ^ 2)

cat("The MSPE for the lm model is: ",lm_mspe,". The MSPE for the GLM model is: ", glm_mspe,
". Finally, the GAM MSPE is: ", gam_mspe)
```

```
The MSPE for the lm model is:  16295.27 . The MSPE for the GLM model is:  565.1429 . Finall
y, the GAM MSPE is:  350.7601
```

The best model is the GAM.

Problem 2 Additive models with the advertising data

The following dataset containts measurements related to the impact of three advertising medias on sales of a product, P. The variables are:

- `youtube` : the advertising budget allocated to YouTube. Measured in thousands of dollars;
- `facebook` : the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- `newspaper` : the advertising budget allocated to a local newspaper. Measured in thousands of dollars.
- `sales` : the value in the i^{th} row of the sales column is a measurement of the sales (in thousands of units) for product P for company i.

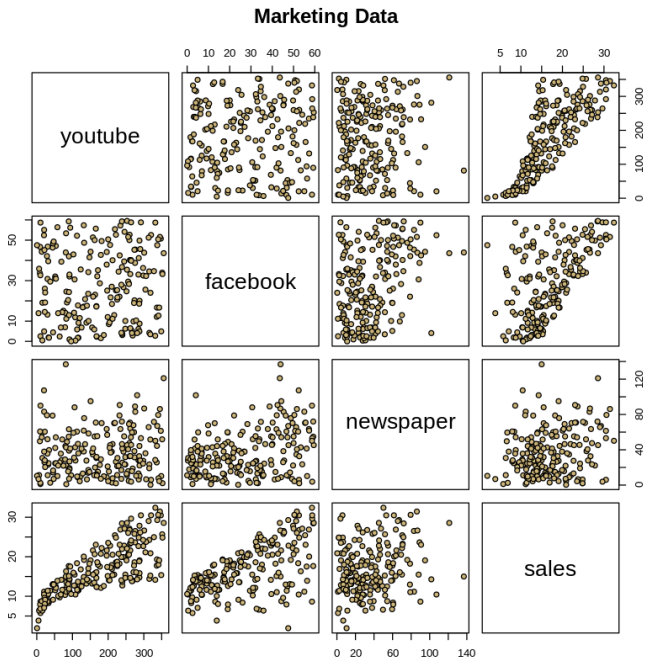
The advertising data treat "a company selling product P" as the statistical unit, and "all companies selling product P" as the population. We assume that the $n = 200$ companies in the dataset were chosen at random from the population (a strong assumption!).

First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

In [9]:
```
# Load in the data
marketing = read.csv("marketing.txt", sep = "")
head(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

A data.frame: 6 × 4

	youtube	facebook	newspaper	sales
	<dbl>	<dbl>	<dbl>	<dbl>
1	276.12	45.36	83.04	26.52
2	53.40	47.16	54.12	12.48
3	20.64	55.08	83.16	11.16
4	181.80	49.56	70.20	22.20
5	216.96	12.96	70.08	15.48
6	10.44	58.68	90.00	8.64



In [10]:
```
set.seed(177) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indicies to be included i
n the training set

train_marketing = marketing[index, ] #set the training set to be the randomly sampled rows of
the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining rows
dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

40 · 4

160 · 4

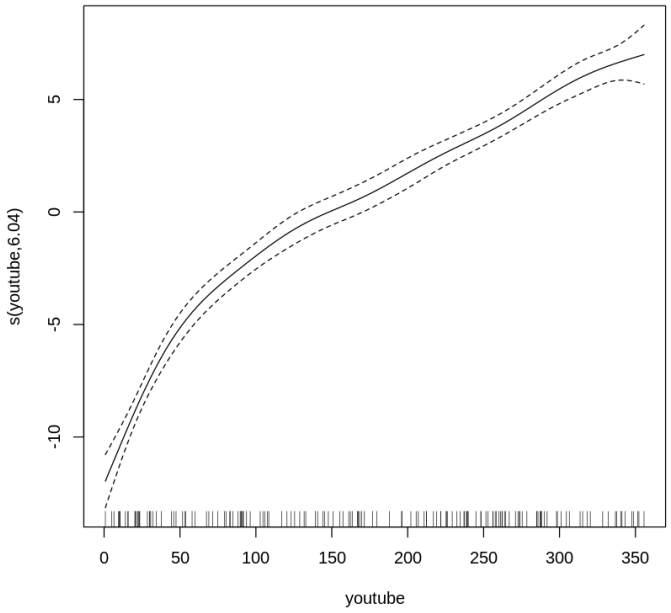
2.(a) Let's try a GAM on the marketing data!

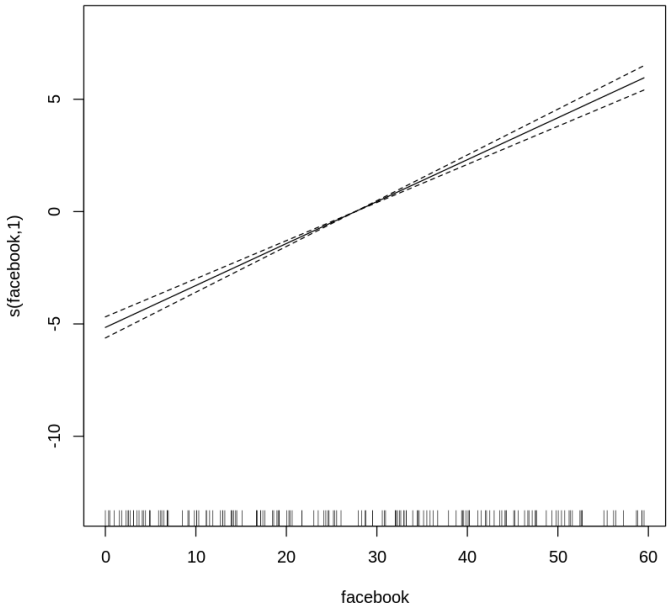
Note that the relationship between `sales` and `youtube` is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. In the last module, we focused on modeling the relationship between `sales` and `youtube`, omitting the other variables. Now it's time to include the additional predictors.

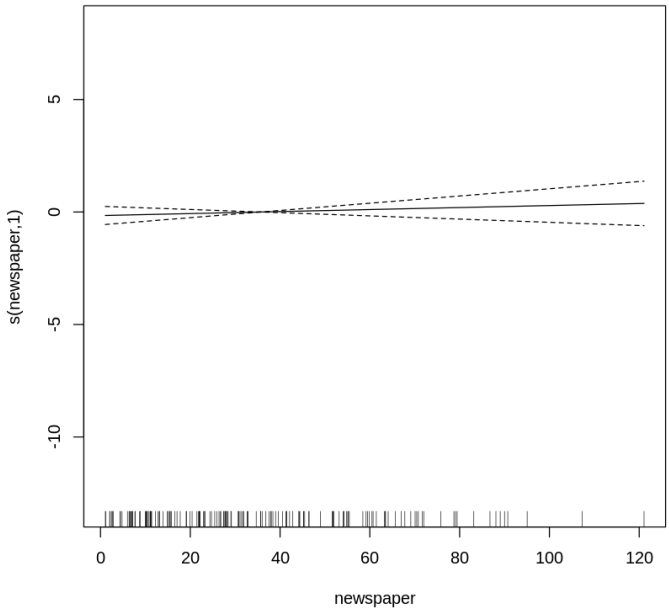
Using the `train_marketing` fit an additive model to the data and store it in `gam_marketing`. Produce the relevant added variable plots using `plot(gam_marketing)`. Comment on the fit of the model.

In [11]:
```
gam_marketing = gam(sales ~ s(youtube) + s(facebook) + s(newspaper), data = train_marketing)
summary(gam_marketing)
plot(gam_marketing)
```

```
Family: gaussian
Link function: identity

Formula:
sales ~ s(youtube) + s(facebook) + s(newspaper)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  16.5743     0.1321   125.5   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Approximate significance of smooth terms:
               edf Ref.df       F p-value
s(youtube)   6.037  7.127 201.356  <2e-16 ***
s(facebook)  1.000  1.000 481.319  <2e-16 ***
s(newspaper) 1.000  1.000   0.593   0.442
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

R-sq.(adj) =  0.929   Deviance explained = 93.3%
GCV = 2.9572  Scale est. = 2.7902    n = 160
```







The edf for Facebook and Newspaper is 1, meaning is basically a linear relationship, this can be confirmed on each of their plots, we can fit a line thru the CI.

2.(b) Semiparametric modeling of the marketing data

Refit the additive model based on your results from 2.(a). That is, if any predictors above should enter linearly, refit the model to reflect that. If any predictors are statistically insignificant, remove them from the model. Store your final model in `semiparametric_marketing` .

In [12]:
```
semiparametric_marketing_first = gam(sales ~ s(youtube) + facebook + newspaper, data = train_
marketing)
summary(semiparametric_marketing_first)
```

```
Family: gaussian
Link function: identity

Formula:
sales ~ s(youtube) + facebook + newspaper

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 11.264013   0.279555  40.293   <2e-16 ***
facebook     0.186582   0.008530  21.874   <2e-16 ***
newspaper    0.004403   0.005811   0.758     0.45
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Approximate significance of smooth terms:
           edf Ref.df     F p-value
s(youtube) 5.9  7.047 202.1  <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

R-sq.(adj) =  0.929   Deviance explained = 93.2%
GCV = 2.9718  Scale est. = 2.8065    n = 160
```

Newspaper shuould be removed from the GAM, so the final model is:

In [13]:
```
semiparametric_marketing = gam(sales ~ s(youtube) + facebook, data = train_marketing)
summary(semiparametric_marketing)
```

```
Family: gaussian
Link function: identity

Formula:
sales ~ s(youtube) + facebook

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 11.352099   0.253895   44.71   <2e-16 ***
facebook     0.189085   0.007848   24.09   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Approximate significance of smooth terms:
             edf Ref.df     F p-value
s(youtube) 5.979  7.127 200.9  <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

R-sq.(adj) =  0.929   Deviance explained = 93.2%
GCV = 2.9438  Scale est. = 2.797     n = 160
```

2.(c) Model comparisons

Now, let's do some model comparisons on the test data. Compute the mean squared prediction error (MSPE) on the `test_marketing` data for the following three models:

- `gam_marketing` from 2.(a)
- `semiparametric_marketing` from 2.(b)
- `lm_marketing`, a linear regression model with `sales` is the response and `youtube` and `facebook` are predictors (fit on the `train_marketing` data).

State which model performs based according to this metric.

In [14]:
```
#mspe for gam

pred_mkt_gam = predict(gam_marketing, newdata = test_marketing, type = "response")
mspe_mkt_gam = mean((test_marketing$sales - pred_mkt_gam)^2)

#mspe for semiparametric

pred_smp_gam = predict(semiparametric_marketing, newdata = test_marketing, type = "response")
mspe_smp_gam = mean((test_marketing$sales - pred_smp_gam)^2)


# mspe for lm

model_lm = lm(sales ~ youtube + facebook, data = train_marketing)
pred_lm = predict(model_lm, newdata = test_marketing)
mspe_lm = mean((test_marketing$sales - pred_lm)^2)

cat("The MSPE for the lm model is: ",mspe_lm,". The MSPE for the GAM model is: ", mspe_mkt_ga
m, ". Finally, the Semmiparametric MSPE is: ", mspe_smp_gam)
```

```
The MSPE for the lm model is:  4.197701 . The MSPE for the GAM model is:  3.438202 . Finall
y, the Semmiparametric MSPE is:  3.297448
```

The semiparametric model has the lowest MSPE. Both are better than the linear mmodel.

In [ ]: